Introdução ao CSS

Estilizando a Web com Elegância

O que é CSS?

- CSS (Cascading Style Sheets)
- Define a aparência e o layout de páginas HTML
- Facilita manutenção e reutilização de estilos
- CSS separa o conteúdo (HTML) da apresentação (estilo)

Como utilizar CSS?

- Inline:

  style="color: blue;"

- Interno:

  <style> dentro do <head>

- Externo:

  Arquivo .css separado (recomendado)

```
body {
   background-color: lightblue;
}
```

Seletores CSS

- Seletores básicos:

  tag
  .classe
  #id

- Seletores avançados:

  div > p
  div p
  p:first-child

- Pseudoclasses e pseudo-elementos

Seletor de tipo (tag)

- Seleciona todos os elementos de um determina tipo (tag HTML).

```
p {
  color: blue;
  font-size: 18px;
}
```

- Todos os <p> da página terão texto azul e tamanho de 18px.

Seletor de classe (.)

- Aplica o estilo a qualquer elemento que possua uma classe específica.
- no *css*:

```
.destaque {
  background-color: yellow;
  font-weight: bold;
}
```

- no *html*:

```
<p class="destaque">Este parágrafo
está em destaque.</p>
```

Seletor de id (#)

- Estiliza um único elemento com um identificador exclusivo.
- no *css*:

```
#cabecalho {
  text-align: center;
  font-size: 24px;
}
```

- no *html*:

```
<h1 id="cabecalho">Título da
Página</h1>
```

Seletor universal (*)

- Aplica estilos a **todos os elementos** da página.
- no *css*:

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}
```

- Isso remove margens e preenchimentos padrão de todos os elementos.

Seletor de descendente ( )

- Aplica o estilo **somente aos elementos dentro de outro elemento específico**.
- no *css*:

```
div p {
  color: red;
}
```

- no *html*:

```
<div>
  <p>Este parágrafo ficará vermelho.</p>
</div>
<p>Este parágrafo não será afetado.</p>
```

Seletor filho (>)

- Seleciona **somente os filhos diretos** de um elemento.
- no *css*:

```
div > p {
  color: green;
}
```

- no *html*:

```
<div>
  <p>Este ficará verde.</p>
  <section>
    <p>Este NÃO será afetado.</p>
  </section>
</div>
```

Seletor de irmão adjacente (+)

- Estiliza o **primeiro elemento que vem logo depois** de um outro específico.
- no *css*:

```
h2 + p {
  color: purple;
}
```

- no *html*:

```
<h2>Título</h2>
<p>Este parágrafo ficará roxo.</p>
<p>Este não será afetado.</p>
```

Seletor de irmãos gerais (~)

- Estiliza **todos os elementos do mesmo nível que vêm depois** de um outro específico.
- no *css*:

```
h2 ~ p {
  font-style: italic;
}
```

- no *html*:

```
<h2>Título</h2>
<p>Este ficará itálico.</p>
<p>Este também ficará itálico.</p>
```

Pseudoclasses

- Aplicam estilos com base no **estado do elemento**.
- no *css*: (estiliza os itens ímpares da lista.)

```
li:nth-child(odd) {
  background-color: #eee;
}
```

- no *css*: (estiliza os itens ímpares da lista.)

```
button:hover {
  background-color: lightblue;
}
```

Pseudoclasses

- Por exemplo, pode ser usado para:
  - Estilizar um elemento quando um usuário move o mouse sobre ele
  - Estilize links visitados e não visitados de forma diferente
  - Estilize um elemento quando ele receber foco
  - Estilo de elementos de formulário válidos/inválidos/obrigatórios/opcionais

Pseudoelementos

- Estilizam **partes específicas** de um elemento.
- no *css*:

```
p::first-letter {
  font-size: 2em;
  color: red;
}
```

https://www.w3schools.com/html/html_emojis.asp

```
h1::before {
  content: "⍰ ";
}
h1::after {
  content: " ⍰  ";
}
```

Pseudoelementos

- Por exemplo, pode ser usado para:
  - Estilizar a primeira letra ou linha de um elemento
  - Insira conteúdo antes ou depois de um elemento
  - Estilizar os marcadores dos itens da lista
  - Estilizar a viewbox atrás de uma caixa de diálogo

Usando emojis no HTML

- Em content, o caractere Unicode deve ser representado com

  \ seguido do código hexadecimal . Ex: \1F9D0
- A notação ⌚ funciona em HTML, mas não no CSS.

https://www.w3schools.com/html/html_emojis.asp
https://symbl.cc/pt/unicode-table/#basic-latin

Cascateamento

- O CSS funciona seguindo um sistema de **cascateamento**
  - **prioridade**, **especificidade** e **ordem de declaração**.

- Algumas propriedades são herdadas automaticamente, enquanto outras não.

Cascateamento

- Exemplo:
- no *html*:

```
<p id="texto" class="destaque">Texto de
Exemplo</p>
```

- no *css*:

```
p { color: blue; } /* Regra no CSS externo */
.destaque { color: green; } /* Classe */
#texto { color: orange; } /* ID */
p { color: purple !important; } /* Regra com
!important */
```

Especificidade – Como o CSS Decide Qual Regra Usar?

Tipo de Seletor	Peso na Especificidade	Exemplo
Inline (`style=""`)	1000	`<p style="color: red;">`
ID (`#meuId`)	100	`#meuId {}`
Classe (`.minhaClasse`), Pseudoclasse (`:hover`)	10	`.minhaClasse {}`
Tag (`p` , `h1`)	1	`p {}`

A regra com o maior peso será aplicada primeiro.

O !important ignora a hierarquia normal e **força a aplicação de um estilo**, independentemente da especificidade.

Herança

- Propriedades Herdáveis
  - ✔ color
  - ✔ font-family
  - ✔ visibility

- Propriedades NÃO Herdáveis
  - margin, padding, border
  - width, height
  - background, display

Herança

- Se quiser **forçar a herança** em uma propriedade não herdável, use **inherit**:
- no *css*:

```
div {
  background-color: inherit; /* Herda a cor do
pai */
}
```

```
p {
  color: initial; /* Volta ao padrão do
navegador */
}
```

Cascateamento e Herança

- O CSS segue uma ordem de prioridade: Inline > Interno > Externo > Padrão
- A última regra aplicada tem mais peso se os seletores forem iguais.
- IDs são mais fortes que classes, que são mais fortes que tags.
- O !important quebra a hierarquia e força a regra.
- Algumas propriedades são herdadas automaticamente, outras não.
- Dica Final: Sempre prefira uma estrutura organizada de seletores e evite !important sempre que possível.

Cores

- Em CSS, podemos definir cores de várias maneiras (https://htmlcolorcodes.com/).
  - **Nomes de Cores:** Exemplo: red, blue, green.
  - **Hexadecimal:** Exemplo: #FF5733.
  - **RGB (Red, Green, Blue):** Exemplo: rgb(255, 87, 51).
  - **RGBA (Red, Green, Blue, Alfa):** Exemplo: rgba(255, 87, 51, 0.5) (com transparência).
  - **HSL (Hue, Saturation, Lightness):** Exemplo: hsl(9, 100%, 60%).
  - **HSL (Hue, Saturation, Lightness, Alfa**):hsla(9, 100%, 60%, 0.5) (com transparência).

Fontes

- Usamos as propriedades de fontes para controlar o estilo e a aparência do texto.
  - **font-family:** Define o tipo de fonte.
  - **font-size:** Define o tamanho da fonte.
  - **font-weight:** Define a espessura da fonte.
  - **text-align:** Define o alinhamento do texto.

Propriedades de Fontes

- **font-family:** Especifica a fonte a ser usada.
  - Exemplo: "Arial", "Times New Roman", "Verdana".

- **font-size:** Define o tamanho da fonte. Pode ser em pixels (px), em (em), ou porcentagem (%).
  - Exemplo: 16px, 1em, 120%.

Propriedade de Fontes

- **font-weight:** Define a espessura da fonte. Valores comuns: normal, bold, ou valores numéricos (100 a 900).
  - Exemplo: normal, bold, 700.

- **text-align:** Alinha o texto em relação ao seu contêiner. Pode ser left, center, right ou justify.
  - Exemplo: left, center, right, justify.

Background

- O **background** no CSS refere-se ao estilo visual de fundo de um elemento.

- Pode incluir **cores, imagens, tamanhos e posições**, entre outros.

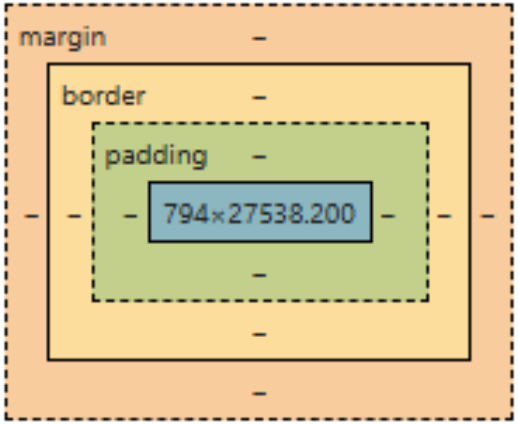
Propriedades do Background

- **background-color**: Define a cor de fundo.
- **background-image**: Define uma imagem de fundo.
- **background-size**: Ajusta o tamanho da imagem de fundo.

Box Model

- O **Box Model** é um conceito importante no layout de páginas web, representando como o conteúdo de um elemento é renderizado.
  - **Content (Conteúdo)**: O conteúdo real do elemento.
  - **Padding (Preenchimento)**: Espaço entre o conteúdo e a borda.
  - **Border (Borda)**: A linha ao redor do elemento.
  - **Margin (Margem)**: Espaço fora da borda, separando o elemento de outros.

Box Model



- **content:** Refere-se ao **conteúdo** real dentro do elemento. O **tamanho do conteúdo** depende do conteúdo textual ou das imagens dentro do elemento.
- **padding: espaçamento entre o conteúdo e a borda** de um elemento.
- **border:** é a linha que envolve o conteúdo e o padding de um elemento.
- **margin: espaçamento externo** ao redor do elemento, separando-o de outros elementos.

Display

- Define como um elemento HTML será exibido na página.

- Ele determina se o elemento ficará em linha (inline), ocupará toda a largura disponível (block), usará um layout flexível (flex), ou se seguirá um sistema de grade (grid), entre outras opções.

Tipos de display

- **block**: Ocupa toda a largura disponível e inicia em uma nova linha.
- **inline**: Ocupa apenas o tamanho do seu conteúdo e permanece na mesma linha.
- **inline-block**: Como inline, mas permite definir altura e largura.
- **flex**: Permite criar layouts flexíveis e responsivos.
- **grid**: Cria um layout baseado em grades.

Tipos de display

- **block**:
  - Sempre inicia em uma nova linha.
  - Ocupa toda a largura disponível.
  - Permite definir altura, largura, margens e paddings.

Tipos de display

- **inline**:
  - Mantém-se na mesma linha de outros elementos inline.
  - Não permite definir altura e largura.

Tipos de display

- **inline-block**:
  - Mantém-se na mesma linha.
  - Permite definir altura e largura.

Tipos de display

- **flex**:
  - Facilita o alinhamento dinâmico dos elementos.
  - Trabalha com espaçamento automático entre itens.

Tipos de display

- **grid**:
  - Divide a página em colunas e linhas.
  - Ideal para layouts complexos.

Tag span

- É um elemento **inline** usado para agrupar partes de um texto ou aplicar estilos a uma parte específica do conteúdo sem afetar o layout ao redor.

- Não tem significado semântico próprio, servindo apenas para estilização ou manipulação via CSS e JavaScript.

Position

- A propriedade position define como um elemento é posicionado na página.

- Existem cinco valores principais:
  - **static** (padrão)
  - **relative**
  - **absolute**
  - **fixed**
  - **sticky**

- Cada um tem um comportamento diferente em relação ao fluxo do documento.

position: static (Padrão)

- O elemento segue o fluxo normal da página, sem alterações de posição.

- O elemento fica na posição normal, sem deslocamento por top, left, right ou bottom.

- Esse é o comportamento padrão de todos os elementos.

position: relative (Relativo à posição original)

- O elemento pode ser deslocado em relação à sua posição original.
- O elemento **ainda ocupa o espaço original no layout**, mas visualmente ele se desloca.
- **top**, **bottom**, **left** e **right** movem o elemento a partir da posição normal.

position: absolute (Fora do fluxo do documento)

- O elemento é removido do fluxo normal e posicionado em relação ao primeiro elemento pai que tenha position: relative, absolute ou fixed.

- Se não houver um ancestral com position: relative, ele será posicionado **em relação ao <html> (viewport).**

- Ele **não ocupa espaço no fluxo normal** do layout.

position: fixed (Fixo na tela, independente do scroll)

- O elemento é fixado na tela e não se move ao rolar a página.

- Ele fica **fixo na tela**, independentemente do rolar da página.

- Muito usado para menus fixos, botões de "voltar ao topo", etc.

position: sticky (Mistura de relative e fixed)

- O elemento se comporta como relative até atingir um determinado ponto da tela, onde então ele se fixa.

- Ele age como um elemento relative até que a página role além do top: 0px, momento em que ele se fixa como fixed.

- Muito usado para **menus fixos ao rolar a página.**

Resumo - position

Tipo	Ocupa espaço normal?	Se move no layout?	Se fixa na tela?
static	☑Sim	✕Não	✕Não
relative	☑Sim	☑Sim (em relação à posição original)	✕Não
absolute	✕Não	☑Sim (em relação ao primeiro ancestral posicionado)	✕Não
fixed	✕Não	☑Sim (em relação à tela)	☑Sim
sticky	☑Sim	☑Sim (até atingir um ponto, onde se fixa)	☑Parcialmente

Dúvidas?